
SMART RC CAR

WITH GY-271 MAGNETOMETER COMPASS

Project Report

Bluetooth Control | USB Control | Digital Compass

Technologies Used

Arduino Uno • HC-05 Bluetooth • GY-271 QMC5883L • L298N Motor Driver • Python

1. Project Overview

This project implements a Smart RC Car controlled wirelessly via Bluetooth and via USB serial connection. The car integrates a GY-271 QMC5883L magnetometer/compass module that continuously broadcasts heading direction and azimuth angle, providing real-time orientation data to the Python controller on a PC.

1.1 Key Features

- Dual control modes: Bluetooth (HC-05) and USB Serial
- Real-time compass heading using GY-271 (QMC5883L)
- 8-direction compass support: N, NE, E, SE, S, SW, W, NW
- Variable speed control: Slow, Medium, Fast
- Live dashboard with compass rose in Python
- Smooth motor control via L298N motor driver

1.2 Components

Component	Model / Value	Qty	Purpose
Microcontroller	Arduino Uno	1	Main controller
Magnetometer	GY-271 (QMC5883L)	1	Compass / heading
Bluetooth Module	HC-05	1	Wireless control
Motor Driver	L298N	1	Drive DC motors
DC Motors	Generic TT Motor	4	Drive wheels
Power Supply	7.4 V LiPo / 9 V	1	Power source
PC / Laptop	Windows with Python 3	1	Controller UI

2. Hardware Wiring

2.1 GY-271 Magnetometer → Arduino (I2C)

The GY-271 communicates via I²C. Connect it as follows:

GY-271 Pin	Arduino Pin	Notes
VCC	3.3 V	Use 3.3 V only — 5 V will damage sensor
GND	GND	Common ground
SDA	A4 (SDA)	I ² C data line
SCL	A5 (SCL)	I ² C clock line

GY-271 Pin	Arduino Pin	Notes
DRDY	—	Optional data-ready pin, leave unconnected

⚠ Important: The QMC5883L is a 3.3 V device. Do NOT connect VCC to the 5 V Arduino pin.

2.2 HC-05 Bluetooth → Arduino (SoftwareSerial)

The HC-05 is wired to SoftwareSerial pins 10 (RX) and 11 (TX):

HC-05 Pin	Arduino Pin	Notes
VCC	5 V	Module requires 5 V
GND	GND	Common ground
TX	Pin 10 (RX-SS)	HC-05 TX → Arduino software RX
RX	Pin 11 (TX-SS)	Arduino software TX → HC-05 RX
EN	—	Leave unconnected (normal mode)
STATE	—	Optional status LED

Note: The HC-05 RX pin is 3.3 V tolerant on some modules; if yours is not, use a voltage divider (1 kΩ + 2 kΩ) on the Arduino TX line.

2.3 L298N Motor Driver → Arduino

L298N Pin	Arduino Pin	Notes
IN1	Pin 2	Left motor direction A
IN2	Pin 3	Left motor direction B
IN3	Pin 4	Right motor direction A
IN4	Pin 5	Right motor direction B
ENA	Pin 6 (PWM)	Left motor speed (analogWrite)
ENB	Pin 9 (PWM)	Right motor speed (analogWrite)
VCC	Battery +	Motor supply voltage (7–12 V)
GND	Battery – / Arduino GND	Common ground
+5V	Arduino 5V (optional)	If on-board 5V regulator used

2.4 Full Wiring Summary

Signal	From	To	Wire Color (suggestion)
I ² C SDA	GY-271 SDA	Arduino A4	Yellow
I ² C SCL	GY-271 SCL	Arduino A5	Green
3.3 V	Arduino 3V3	GY-271 VCC	Red
GND	Arduino GND	GY-271 GND	Black
BT TX→RX	HC-05 TX	Arduino Pin 10	White
BT RX←TX	Arduino P11	HC-05 RX	Blue
5 V	Arduino 5V	HC-05 VCC	Red
Motor IN1–4	Arduino 2–5	L298N IN1–4	Orange
Motor ENA	Arduino 6	L298N ENA	Purple
Motor ENB	Arduino 9	L298N ENB	Brown

3. Bluetooth Control

In Bluetooth mode, the PC connects to the HC-05 via a virtual COM port (e.g., COM7 or COM8). The Python script sends single-character commands and reads compass data — all through the same wireless serial link.

3.1 Arduino Code — Bluetooth + Compass

Upload this sketch to the Arduino. It listens for commands on both the HC-05 SoftwareSerial port and the USB serial, drives the motors, reads the GY-271, and broadcasts the heading back over Bluetooth.

```
#include <SoftwareSerial.h>
#include <QMC5883LCompass.h>

// Bluetooth on pins 10 (RX) / 11 (TX)
SoftwareSerial bluetooth(10, 11);
QMC5883LCompass compass;

// L298N motor pins
#define IN1 2
#define IN2 3
#define IN3 4
#define IN4 5
#define ENA 6
#define ENB 9

int L_SPEED    = 220;
int R_SPEED    = 220;
int TURN_SPEED = 170;

void setup() {
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
```

```
pinMode(IN3,OUTPUT); pinMode(IN4,OUTPUT);
pinMode(ENA,OUTPUT); pinMode(ENB,OUTPUT);
Serial.begin(9600);
bluetooth.begin(9600);
compass.init();
stopCar();
Serial.println("SMART RC CAR READY");
bluetooth.println("SMART RC CAR READY");
}

void loop() {
  char cmd = 0;
  if (bluetooth.available()) cmd = bluetooth.read();
  if (Serial.available()) cmd = Serial.read();
  if (cmd) control(cmd);

  // --- Compass reading ---
  compass.read();
  int angle = compass.getAzimuth();
  String dir;
  if (angle < 45 || angle >= 315) dir = "NORTH";
  else if (angle < 135) dir = "EAST";
  else if (angle < 225) dir = "SOUTH";
  else dir = "WEST";

  // Broadcast to both USB and Bluetooth
  Serial.print("ANGLE: "); Serial.print(angle);
  Serial.print(" "); Serial.println(dir);
  bluetooth.print("ANGLE: "); bluetooth.print(angle);
  bluetooth.print(" "); bluetooth.println(dir);
  delay(300);
}

void control(char c) {
  switch(c) {
    case 'F': case 'f': forwardCar(); break;
    case 'B': case 'b': backCar(); break;
    case 'L': case 'l': leftCar(); break;
    case 'R': case 'r': rightCar(); break;
    case 'G': case 'g': rotateLeft(); break;
    case 'I': case 'i': rotateRight(); break;
    case 'S': case 's': stopCar(); break;
    case '1': L_SPEED = R_SPEED = 80; break;
    case '2': L_SPEED = R_SPEED = 150; break;
    case '3': L_SPEED = R_SPEED = 220; break;
  }
}

void forwardCar() { digitalWrite(IN1,HIGH); digitalWrite(IN2,LOW);
                  digitalWrite(IN3,HIGH); digitalWrite(IN4,LOW);
                  analogWrite(ENA,L_SPEED); analogWrite(ENB,R_SPEED); }
void backCar()   { digitalWrite(IN1,LOW); digitalWrite(IN2,HIGH);
                  digitalWrite(IN3,LOW); digitalWrite(IN4,HIGH);
                  analogWrite(ENA,L_SPEED); analogWrite(ENB,R_SPEED); }
void leftCar()   { digitalWrite(IN1,LOW); digitalWrite(IN2,LOW);
                  digitalWrite(IN3,HIGH); digitalWrite(IN4,LOW);
                  analogWrite(ENA,0); analogWrite(ENB,TURN_SPEED); }
void rightCar()  { digitalWrite(IN1,HIGH); digitalWrite(IN2,LOW);
                  digitalWrite(IN3,LOW); digitalWrite(IN4,LOW);
                  analogWrite(ENA,TURN_SPEED); analogWrite(ENB,0); }
void rotateLeft() { digitalWrite(IN1,LOW); digitalWrite(IN2,HIGH);
                  digitalWrite(IN3,HIGH); digitalWrite(IN4,LOW); }
```

```
        analogWrite(ENA,TURN_SPEED); analogWrite(ENB,TURN_SPEED);
    }
    void rotateRight() { digitalWrite(IN1,HIGH); digitalWrite(IN2,LOW);
                        digitalWrite(IN3,LOW); digitalWrite(IN4,HIGH);
                        analogWrite(ENA,TURN_SPEED); analogWrite(ENB,TURN_SPEED);
    }
    void stopCar()      { digitalWrite(IN1,LOW); digitalWrite(IN2,LOW);
                        digitalWrite(IN3,LOW); digitalWrite(IN4,LOW);
                        analogWrite(ENA,0); analogWrite(ENB,0); }
```

3.2 Python Bluetooth Controller

Install dependencies first: `pip install pyserial keyboard`

```
import serial, keyboard, time, sys

# ----- SETTINGS -----
COM_PORT = "COM8" # HC-05 Bluetooth virtual COM port
BAUD      = 9600

# ----- CONNECT -----
print("Connecting via Bluetooth...")
try:
    bt = serial.Serial(COM_PORT, BAUD, timeout=0.1)
    time.sleep(2)
    print("Bluetooth Connected - SMART RC CAR STARTED\n")
except:
    print("Bluetooth not connected. Check COM port.")
    sys.exit()

print("""
===== CONTROL =====
UP      : Forward    DOWN  : Backward
LEFT    : Left       RIGHT : Right
G       : Rotate L   I     : Rotate R
1/2/3   : Slow/Med/Fast  SPACE : Stop
ESC     : Exit
===== """)

last_cmd = ""

def send(cmd):
    global last_cmd
    if cmd != last_cmd:
        bt.write(cmd.encode())
        print("CMD:", cmd)
        last_cmd = cmd

while True:
    # ---- Read compass over Bluetooth ----
    while bt.in_waiting:
        data = bt.readline().decode(errors="ignore").strip()
        if "ANGLE" in data:
            print("Compass:", data)

    # ---- Keyboard control ----
    if keyboard.is_pressed("up"):   send("F")
    elif keyboard.is_pressed("down"): send("B")
    elif keyboard.is_pressed("left"): send("L")
    elif keyboard.is_pressed("right"): send("R")
    elif keyboard.is_pressed("g"):   send("G")
```

```
elif keyboard.is_pressed("i"):    send("I")
elif keyboard.is_pressed("1"):    send("1")
elif keyboard.is_pressed("2"):    send("2")
elif keyboard.is_pressed("3"):    send("3")
elif keyboard.is_pressed("space"): send("S")
elif keyboard.is_pressed("esc"):  send("S"); break
else:                             send("S")
time.sleep(0.05)

bt.close()
print("Car stopped.")
```

3.3 Bluetooth Command Reference

Key	Command Sent	Action
↑ Up Arrow	F	Forward
↓ Down Arrow	B	Backward
← Left Arrow	L	Turn Left
→ Right Arrow	R	Turn Right
G	G	Rotate Left (tank)
I	I	Rotate Right (tank)
1	1	Speed: Slow (PWM 80)
2	2	Speed: Medium (PWM 150)
3	3	Speed: Fast (PWM 220)
Space	S	Stop
ESC	S → Exit	Stop & quit

4. USB (Serial) Control

In USB mode, the Arduino is connected directly to the PC via the USB cable (no Bluetooth required). The Python script opens the Arduino's COM port, sends movement commands, and receives compass data over the same USB link. This is ideal for bench testing or indoor environments.

4.1 Arduino Code — USB + Compass

This sketch accepts commands through the native USB serial port and outputs 8-direction compass data.

```
#include <Wire.h>
#include <QMC5883LCompass.h>

QMC5883LCompass compass;

// L298N motor pins
```

```
#define IN1 2
#define IN2 3
#define IN3 4
#define IN4 5
#define ENA 6
#define ENB 9

int speedCar = 180;

void setup() {
  pinMode(IN1,OUTPUT); pinMode(IN2,OUTPUT);
  pinMode(IN3,OUTPUT); pinMode(IN4,OUTPUT);
  pinMode(ENA,OUTPUT); pinMode(ENB,OUTPUT);
  Serial.begin(9600);
  Wire.begin();
  compass.init();
  stopCar();
  Serial.println("USB RC CAR + COMPASS READY");
}

void loop() {
  // --- USB command ---
  if (Serial.available()) {
    char cmd = Serial.read();
    if (cmd=='F') forward();
    if (cmd=='B') backward();
    if (cmd=='L') left();
    if (cmd=='R') right();
    if (cmd=='S') stopCar();
    if (cmd=='1') speedCar = 80;
    if (cmd=='2') speedCar = 150;
    if (cmd=='3') speedCar = 220;
  }

  // --- 8-direction compass reading ---
  compass.read();
  int x = compass.getX();
  int y = compass.getY();
  int angle = atan2(y, x) * 180 / PI;
  if (angle < 0) angle += 360;

  String direction;
  if (angle >= 337 || angle < 22) direction = "NORTH";
  else if (angle < 67) direction = "NORTH EAST";
  else if (angle < 112) direction = "EAST";
  else if (angle < 157) direction = "SOUTH EAST";
  else if (angle < 202) direction = "SOUTH";
  else if (angle < 247) direction = "SOUTH WEST";
  else if (angle < 292) direction = "WEST";
  else direction = "NORTH WEST";

  Serial.print("ANGLE:");
  Serial.print(angle);
  Serial.print(" DIR:");
  Serial.println(direction);
  delay(200);
}

void forward() { digitalWrite(IN1,HIGH); digitalWrite(IN2,LOW);
                 digitalWrite(IN3,HIGH); digitalWrite(IN4,LOW);
                 analogWrite(ENA,speedCar); analogWrite(ENB,speedCar); }

void backward() { digitalWrite(IN1,LOW); digitalWrite(IN2,HIGH);
                  digitalWrite(IN3,LOW); digitalWrite(IN4,HIGH); }
```

```
        analogWrite(ENA,speedCar); analogWrite(ENB,speedCar); }
void left()    { digitalWrite(IN1,LOW); digitalWrite(IN2,LOW);
                digitalWrite(IN3,HIGH); digitalWrite(IN4,LOW);
                analogWrite(ENA,0); analogWrite(ENB,speedCar); }
void right()   { digitalWrite(IN1,HIGH); digitalWrite(IN2,LOW);
                digitalWrite(IN3,LOW); digitalWrite(IN4,LOW);
                analogWrite(ENA,speedCar); analogWrite(ENB,0); }
void stopCar() { digitalWrite(IN1,LOW); digitalWrite(IN2,LOW);
                digitalWrite(IN3,LOW); digitalWrite(IN4,LOW);
                analogWrite(ENA,0); analogWrite(ENB,0); }
```

4.2 Python USB Controller

```
import serial, keyboard, time

PORT = "COM5"      # Arduino USB COM port
BAUD = 9600

arduino = serial.Serial(PORT, BAUD, timeout=0.1)
time.sleep(2)

print("RC CAR USB CONTROL + COMPASS")
print("Arrow keys to drive | ESC to exit")

last_cmd = ""

def send(cmd):
    global last_cmd
    if cmd != last_cmd:
        arduino.write(cmd.encode())
        last_cmd = cmd
        print("CMD:", cmd)

while True:
    # ---- Read compass data ----
    if arduino.in_waiting:
        data = arduino.readline().decode(errors="ignore").strip()
        if data:
            print(data)

    # ---- Key control ----
    if keyboard.is_pressed("up"):   send("F")
    elif keyboard.is_pressed("down"): send("B")
    elif keyboard.is_pressed("left"): send("L")
    elif keyboard.is_pressed("right"): send("R")
    elif keyboard.is_pressed("1"):   send("1")
    elif keyboard.is_pressed("2"):   send("2")
    elif keyboard.is_pressed("3"):   send("3")
    elif keyboard.is_pressed("space"): send("S")
    elif keyboard.is_pressed("esc"):  send("S"); break
    else:                             send("S")
    time.sleep(0.08)

arduino.close()
print("CAR STOPPED")
```

4.3 Dual-Port Mode (BT + USB simultaneously)

For maximum flexibility, the combined sketch (Section 3.1) already accepts commands from both the Bluetooth SoftwareSerial port AND the USB serial port simultaneously. The Python

dual-port script below opens two COM ports: BT for driving, USB for reading compass data with higher precision.

```
import serial, keyboard, time, threading, os, sys

BT_PORT = "COM8" # HC-05 Bluetooth
USB_PORT = "COM5" # Arduino USB
BAUD = 9600

# Connect Bluetooth
try:
    bt = serial.Serial(BT_PORT, BAUD, timeout=0.1)
    time.sleep(2)
    print("Bluetooth OK:", BT_PORT)
except Exception as e:
    print("BT Error:", e); sys.exit(1)

# Connect USB (optional - compass only)
usb, usb_ok = None, False
try:
    usb = serial.Serial(USB_PORT, BAUD, timeout=1)
    time.sleep(2)
    usb_ok = True
    print("USB OK:", USB_PORT)
except:
    print("USB not connected - compass disabled")

compass_angle = "---"
compass_dir = "---"
last_cmd = ""
running = True

def read_compass():
    global compass_angle, compass_dir
    while running:
        try:
            if usb_ok and usb.in_waiting > 0:
                line = usb.readline().decode('utf-8', errors='ignore').strip()
                if line.startswith("ANGLE:"):
                    parts = line.replace("ANGLE:", "").strip().split()
                    if len(parts) >= 2:
                        compass_angle = parts[0] + " deg"
                        compass_dir = parts[1]
        except: pass
        time.sleep(0.05)

threading.Thread(target=read_compass, daemon=True).start()

def send(cmd):
    global last_cmd
    if cmd != last_cmd:
        bt.write(cmd.encode())
        last_cmd = cmd

status = "STOPPED"
tick = 0

try:
    while True:
        if keyboard.is_pressed('up'): send('F'); status = "FORWARD"
        elif keyboard.is_pressed('down'): send('B'); status = "BACKWARD"
        elif keyboard.is_pressed('left'): send('L'); status = "TURN LEFT"
```

```
elif keyboard.is_pressed('right'): send('R'); status = "TURN RIGHT"
elif keyboard.is_pressed('g'):     send('G'); status = "ROTATE LEFT"
elif keyboard.is_pressed('i'):     send('I'); status = "ROTATE RIGHT"
elif keyboard.is_pressed('space'): send('S'); status = "STOPPED"
elif keyboard.is_pressed('1'):     send('1'); status = "SLOW"
elif keyboard.is_pressed('2'):     send('2'); status = "MEDIUM"
elif keyboard.is_pressed('3'):     send('3'); status = "FAST"
elif keyboard.is_pressed('esc'):    send('S'); break
else:                               send('S'); status = "STOPPED"

tick += 1
if tick % 5 == 0:
    os.system('cls')
    print(f"Status: {status}")
    print(f"Angle  : {compass_angle}")
    print(f"Dir    : {compass_dir}")
    time.sleep(0.08)
except Exception as e:
    print("Error:", e)

running = False
bt.write(b'S')
bt.close()
if usb_ok: usb.close()
print("Car stopped. Goodbye!")
```

5. Digital Compass — Standalone Code

This section provides the complete standalone compass sketch for the GY-271 (QMC5883L). Use it to verify the sensor works correctly before integrating it into the full RC car project.

5.1 Required Arduino Library

Install via Arduino IDE → Library Manager → search "QMC5883LCompass" by MPrograms.

Library
QMC5883LCompass by MPrograms — available in Arduino Library Manager

5.2 Standalone Compass Arduino Sketch

This sketch reads azimuth (0–360°) and 8-point cardinal direction with smoothing enabled. Upload and open Serial Monitor at 9600 baud.

```
#include <Wire.h>
#include <QMC5883LCompass.h>

QMC5883LCompass compass;

void setup() {
    Serial.begin(9600);
    compass.init();

    // Enable smoothing over 10 samples,
    // true = 8 directions (N, NE, E, SE, S, SW, W, NW)
```

```
compass.setSmoothing(10, true);
}

void loop() {
  // Read the sensor
  compass.read();

  // Get azimuth angle (0 - 360 degrees)
  int azimuth = compass.getAzimuth();

  // Get 2-character direction string (e.g., "N", "NE", "E")
  char direction[3];
  compass.getDirection(direction, azimuth);

  // Print to Serial Monitor
  Serial.print("Azimuth: ");
  Serial.print(azimuth);
  Serial.print(" deg | Direction: ");
  Serial.println(direction);

  delay(500); // 0.5 s between readings
}
```

5.3 Example Serial Monitor Output

```
Azimuth: 9 deg | Direction: N
Azimuth: 43 deg | Direction: NE
Azimuth: 91 deg | Direction: E
Azimuth: 179 deg | Direction: S
Azimuth: 273 deg | Direction: W
```

5.4 Direction Angle Chart

Direction	Abbreviation	Angle Range (°)
North	N	338 – 360 and 0 – 22
North-East	NE	23 – 67
East	E	68 – 112
South-East	SE	113 – 157
South	S	158 – 202
South-West	SW	203 – 247
West	W	248 – 292
North-West	NW	293 – 337

5.5 Calibration Tips

- After power-on, slowly rotate the sensor through all axes (figure-8 motion) for 10–15 seconds to allow the QMC5883L to auto-calibrate.
- Keep metal objects, motors, and high-current wires at least 5 cm from the sensor to avoid magnetic interference.
- Mount the GY-271 horizontally and level for most accurate readings.

- The `setSmoothing(10, true)` call averages the last 10 readings; increase the number (max 10) to reduce jitter.

6. Troubleshooting

Symptom	Likely Cause	Fix
Compass reads 0 always	GY-271 VCC on 5V	Move to 3.3 V pin
'0x00 not found' error	Wrong I ² C address	Run I ² C scanner sketch
Bluetooth won't pair	Default PIN not set	Default PIN: 1234 or 0000
COM port busy error	Arduino IDE Monitor open	Close Serial Monitor first
Car moves wrong way	Motor polarity swapped	Swap IN1/IN2 or IN3/IN4
Noisy compass near motors	EMI from motor wires	Move GY-271, add decoupling